

RTL, SIMULATION, SYNTHESIS & FPGA

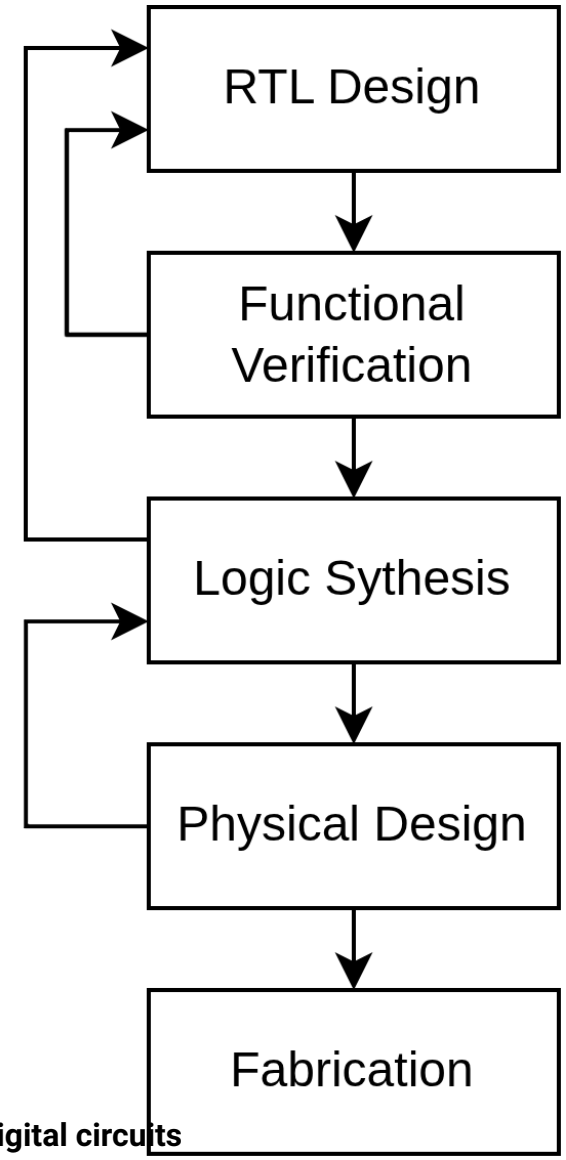
From design specifications to digital circuits

Daniel Arevalos

Munich University of Applied Sciences HM

Where are we ?

- Previous session we presented an overview of the chip design flow.

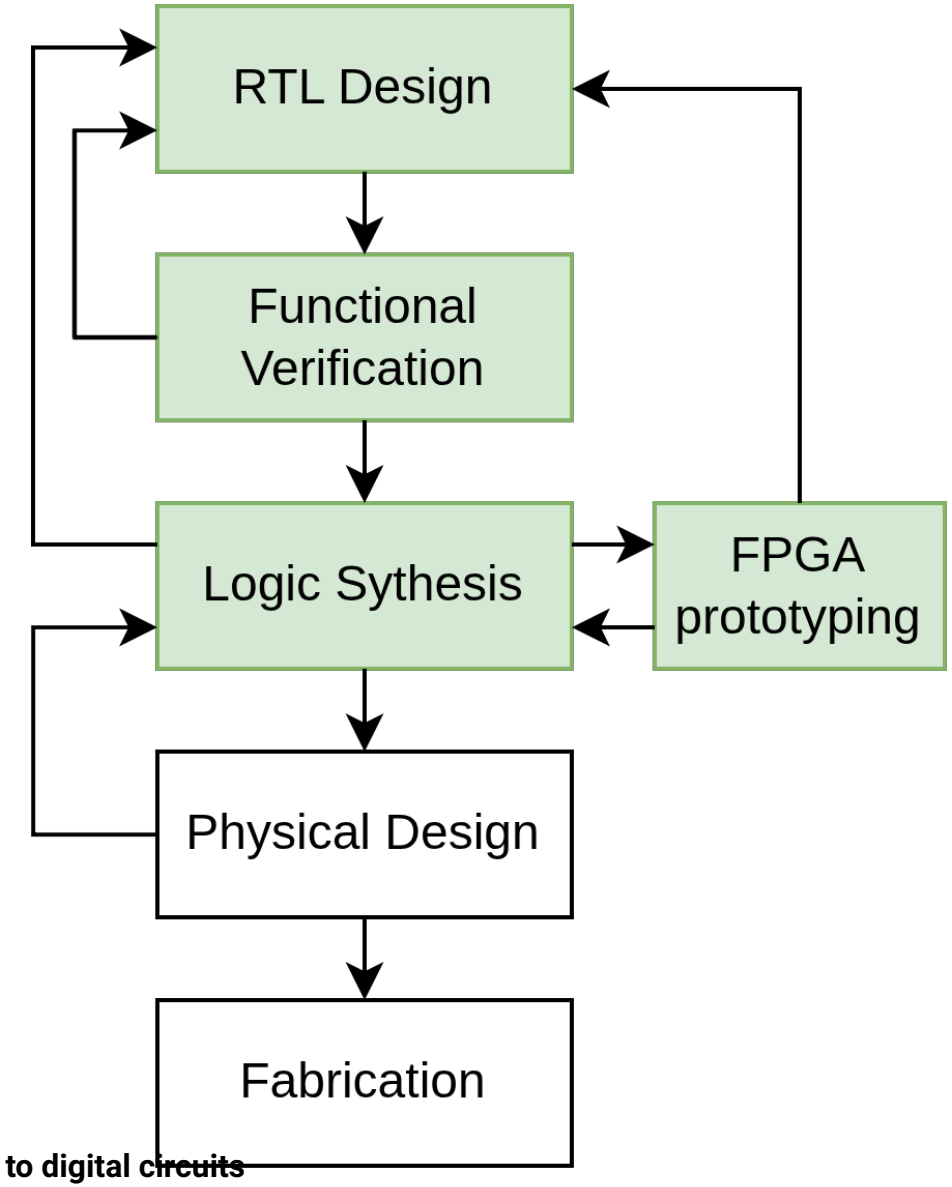


More information of: Digital Design Flow

Where are we ?

- Previous session we presented an overview of the chip design flow.
- This session we are going to see in more details:
  - RTL Design
  - Functional Verification
  - Logic synthesis
  - FPGA Prototyping

More information of: Digital Design Flow



From Specification to Hardware

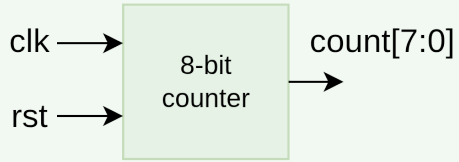
1. Specifications

Example:
8-bit counter

Requirements:
- Counts from 0 to 255
- Increments on each clock cycle
- Synchronous reset
- 8-bit output

- Define what the hardware must do
- Identify inputs, outputs and constraints
- Describe the expectede behavior

2. Architecture



- Devide the design into functional blocks
- Define interfaces and signals
- Specify how the blocks are connected

3. RTL Design

```
module counter (
    input logic clk,
    input logic rst,
    output logic [7:0] count
);
always_ff @(posedge clk) begin
    if (rst) count <= 8'd0;
    else count <= count + 8'd1;
end
endmodule
```

- Describe the hardware's behavior and structure
- Use hierarchy to build larger designs

More information of: RTL Design

Thinking in Hardware: Software vs Hardware

RTL (Register Transfer Level) is a hardware design abstraction that describes how data is processed by combinational logic and stored or transferred between registers.

Software

```
int a = 5;
int b = 3;

int c = a + b;
int d = a - b;
```

```
Program instructions

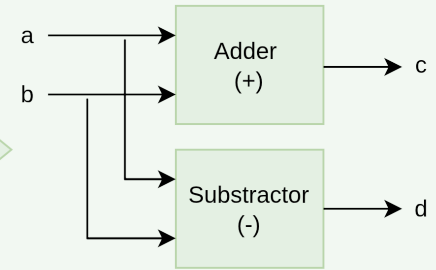
1. 0011 0100 1000 1011
2. 0011 0100 1000 1011
3. ...
```

Time

- Describes a sequence of instructions to be executed
- The same hardware (processor) executes different instructions

RTL

```
module example (
    input logic [7:0] a,
    input logic [7:0] b,
    output logic [7:0] c,
    output logic [7:0] d,
);
assign c = a + b;
assign d = a - b;
endmodule
```



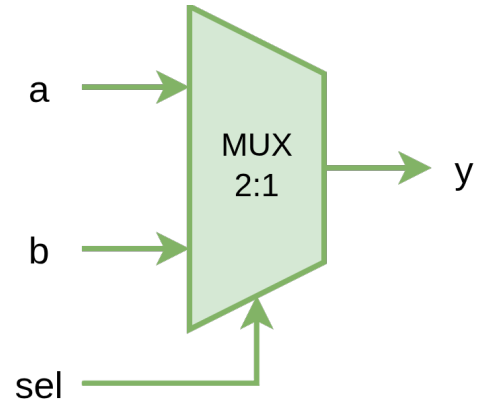
- Independent hardware blocks operate at the same time
- Operations are implemented using hardware resources (adders, substractors, registers, LUTs, etc)

More information of: RTL Design

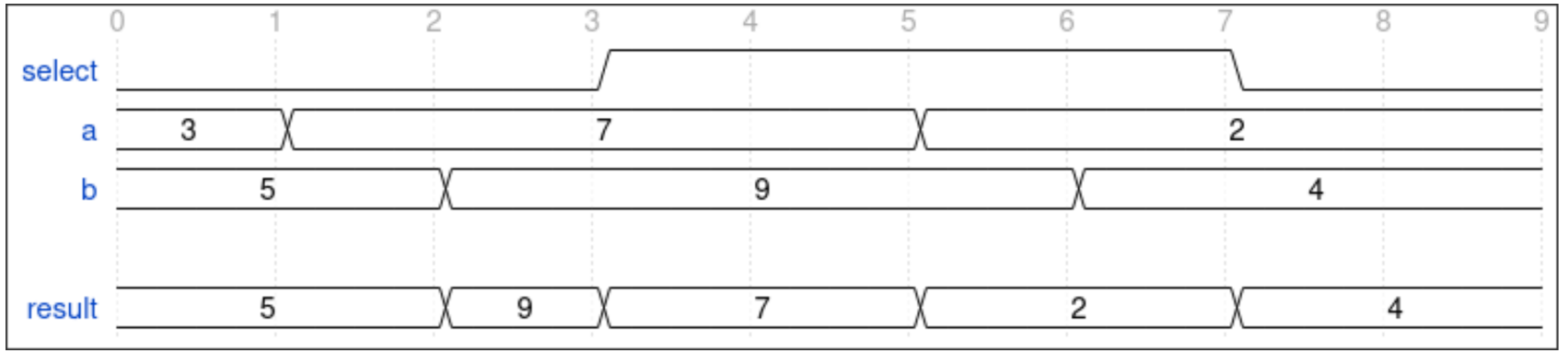
Combinational Logic

- Outputs depend only on the current inputs
- No internal state or memory
- No clock required
- Outputs change after a propagation delay when inputs change

sel	y
1	a
0	b

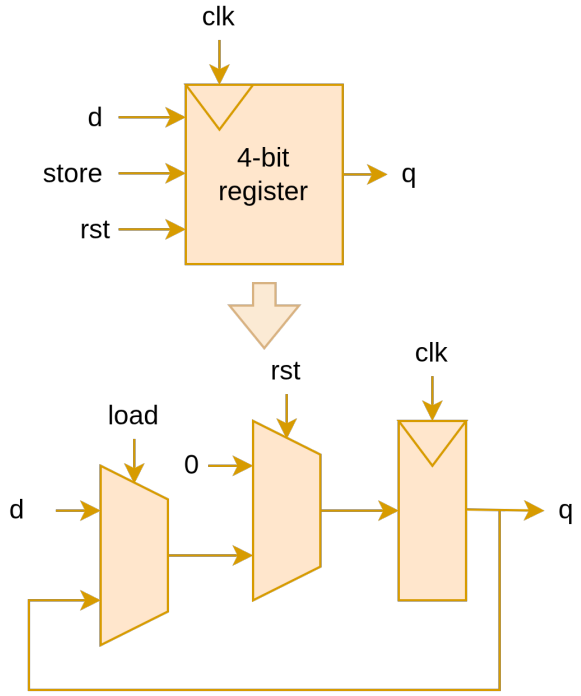


More information of: RTL Design

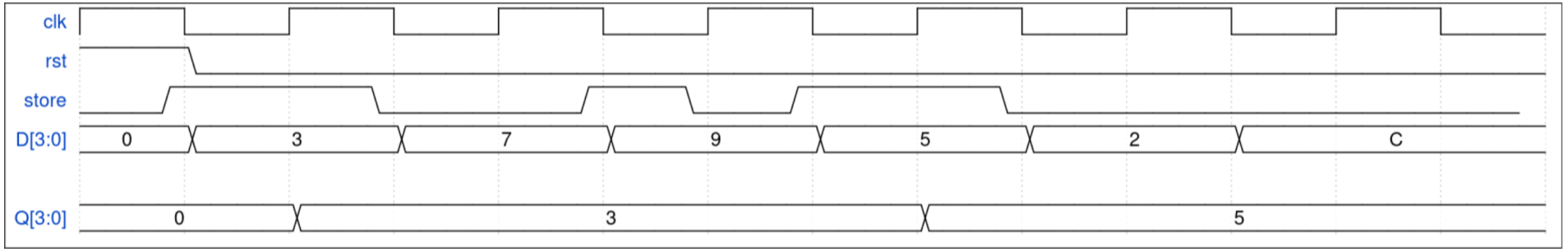


Sequential Logic

- Output depend on the current inputs and the stored state
- Registers store information using flip-flops
- State updated are typically synchronized by a clock
- A reset initializes the circuit to a known state



More information of: RTL Design

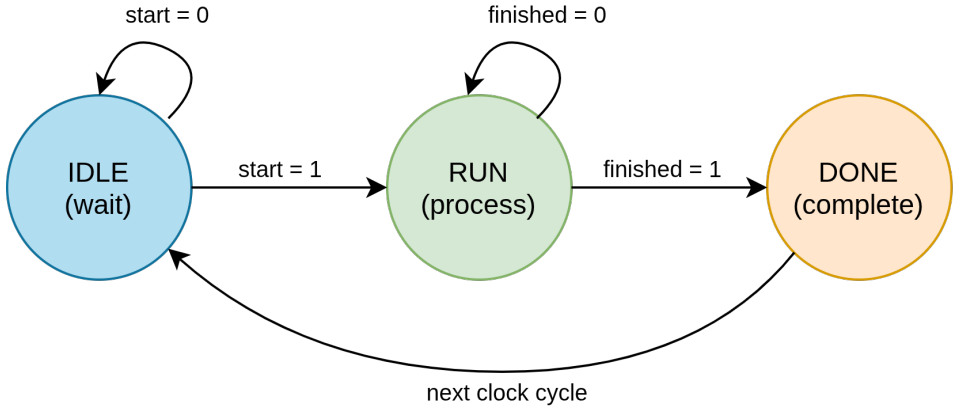


Finite State Machines (FSMs)

FSM (Finite State Machine) is a digital control structure that operates through a finite set of states.

- Combinational logic computes the next state and outputs
- A register stores the current state
- State transitions depend on the current state and inputs

More information of: FSM



State	Behavior	Transition (next state)
IDLE	Wait for start signal	start = 1 → RUN start = 0 → IDLE
RUN	Execute operation	finished = 1 → DONE finished = 0 → IDLE
DONE	Signal completion	Next clock cycle → IDLE

Hardware Description Lenguages (HDLs)

- Text-based lenguages to describe digital circuits
- Software is written using programming languages. Hardware is described using hardware description languages
- Enable simulation and synthesis of digital designs
- Some HDLs:
  - VHDL
  - Verilog
  - SystemVerilog
  - Spade

More information of: HDLs

SystemVerilog

- Module Declaration: Name, parameters, and I/O ports.
- Module Body: Contains the internal hardware description.
- Internal Signals: Data and connections within the module.
- Module Instantiation: Reusing and connecting hardware blocks.
- Combinational/Sequential Logic: Continuous assignments describing logic operations or sequential logic.

More Information on: Refresher on SystemVerilog

```
module top #(
parameter int Width = 16
) (
input logic clk_i,
input logic rst_ni,
input logic mode_i,
input logic [Width-1:0] data_in_i,
output logic [Width-1:0] result_o
);

// Declare signals to be used in the module
logic [Width-1:0] first, second;
logic [Width-1:0] combine_and, combine_or;

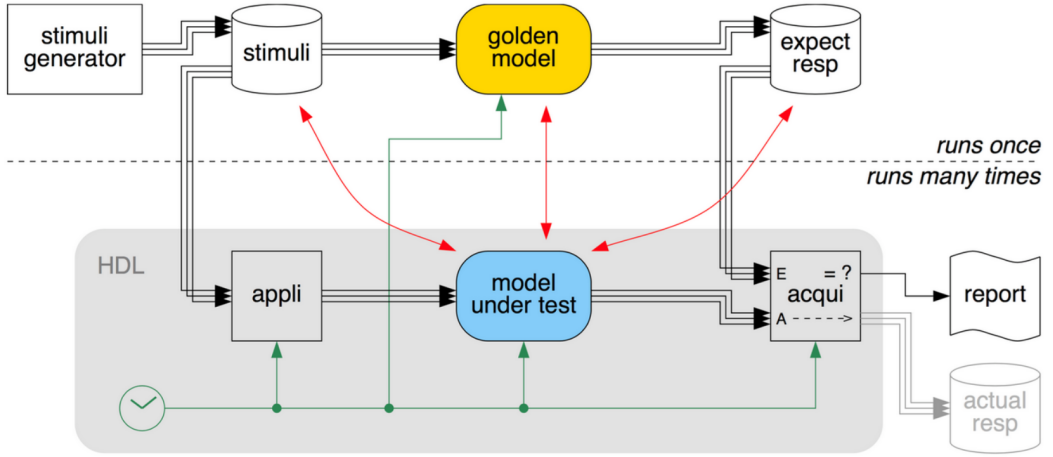
// instantiate two blocks with different names i_reg_1 and frank
ffs #(.Width(Width)) i_reg_1 (
.clk_i(clk_i), .rst_ni(rst_ni), .in_i(data_in_i), .out_o(first));
ffs #(.Width(Width)) frank (
.clk_i(clk_i), .rst_ni(rst_ni), .in_i(first), .out_o(second));

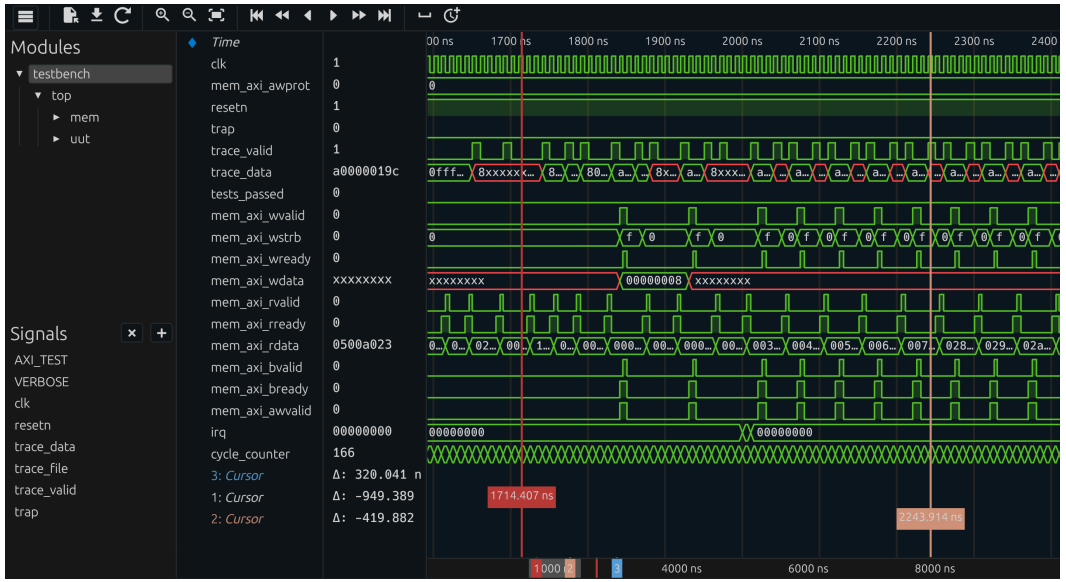
// combine outputs
assign combine_and = first & second ;
assign combine_or = first | second ;
// assign result
assign result_o = mode_i ? combine_and : combine_or ;

endmodule // top
```

Functional Verification (Simulation)

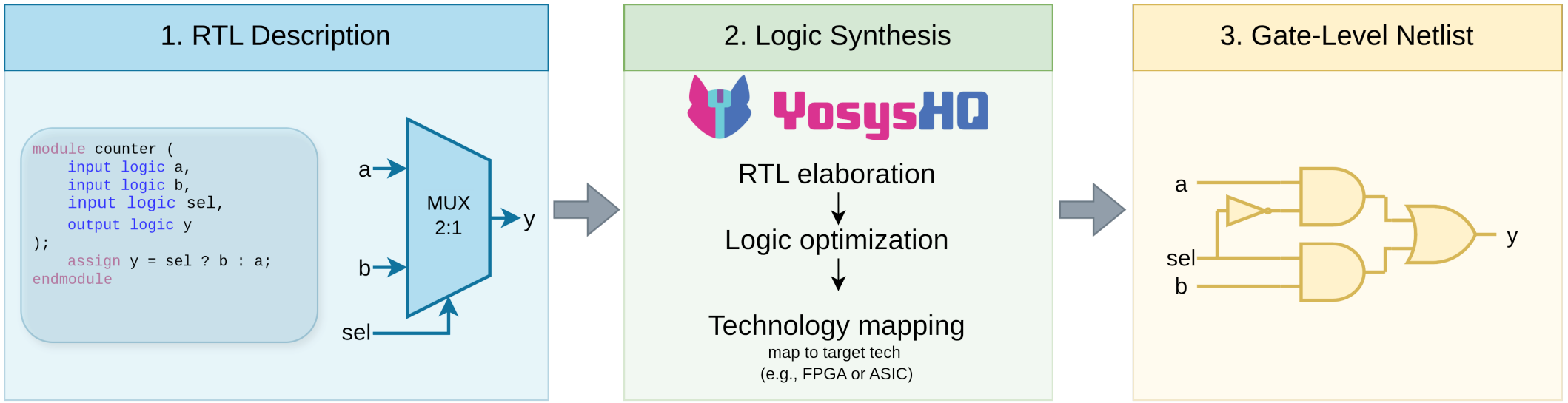
- Validates RTL functionality against the design specifications.
- Simulation checks behavior using testbenches and input stimuli.
  - Applies stimuli to the Design Under Test (DUT) using testbenches.
  - Compares DUT outputs against a golden reference model.
  - Detects functional mismatches and reports errors.





More Information on: Simulation

Synthesis - Overview

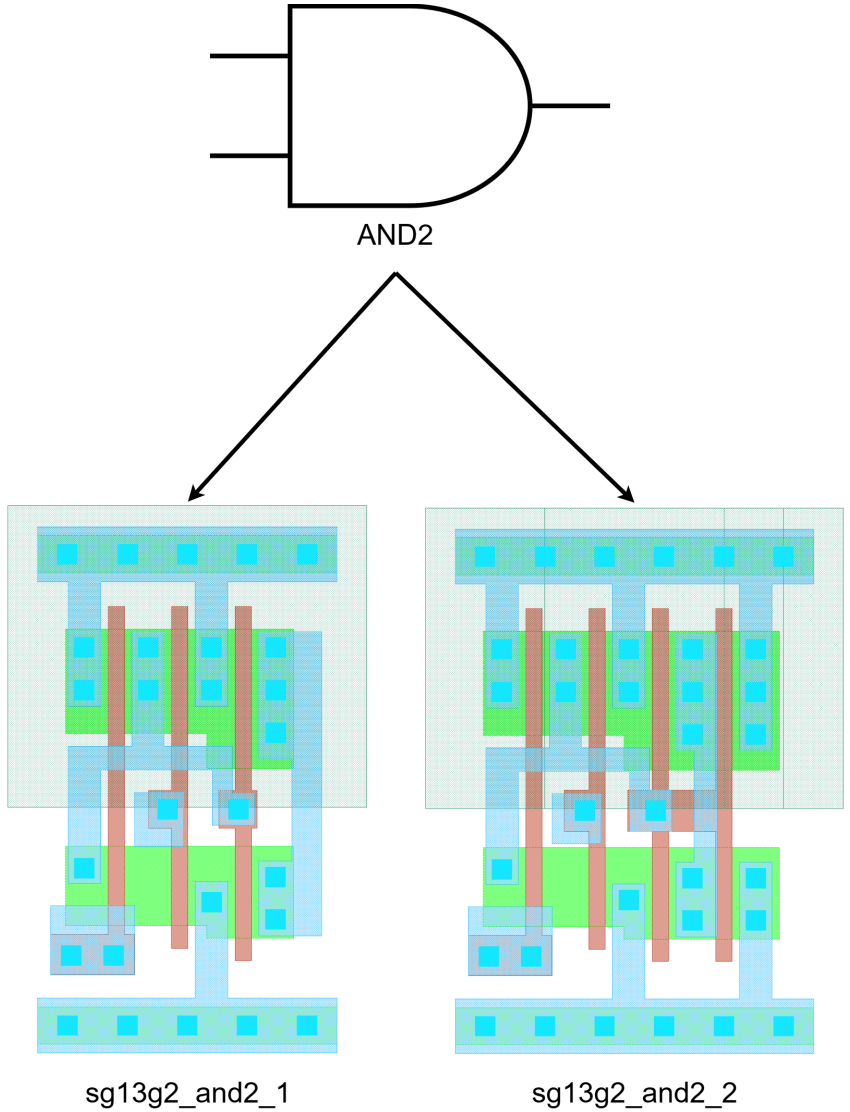


More information on: Synthesis

Synthesis - Standard Cells

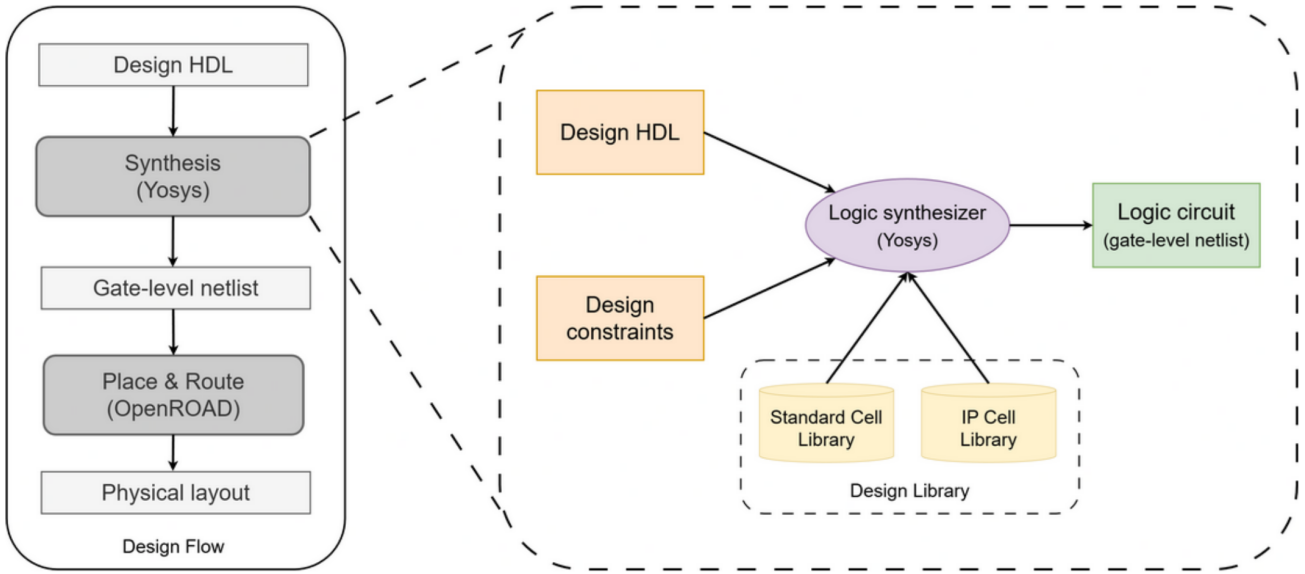
- Pre-designed and pre-characterized logic building blocks.
- Provided by the foundry or PDK as a standard cell library.
- Characterized for timing, power, and area.
- Used by synthesis tools to map RTL into hardware.

More Information on: Synthesis



Synthesis - Inputs & Outputs

- Inputs: RTL design and design constraints.
- Uses standard cell and IP libraries for mapping.
- Translates RTL into a gate-level netlist.
- Preserves functional behavior while meeting timing and area constraints.



More Information on: Synthesis

Synthesis - How it looks in Practice?

```
module fulladd (
  input [3:0] a,
  input [3:0] b,
  input c_in,
  output c_out,
  output [3:0] sum);

  assign {c_out, sum} = a + b + c_in;
endmodule
```

RTL code

More Information on: Synthesis

```
module fulladd(a, b, c_in, c_out, sum);
  wire _00_;
  wire _01_;
  wire _02_;
  ...
  input [3:0] a;
  wire [3:0] a;
  ...
  assign _14_ = _07_ & ~(_13_);
  assign _15_ = _06_ & ~(_14_);
  assign c_out = _15_ | _04_;
  assign sum[0] = ~(_10_ ^ c_in);
  assign sum[1] = ~(_12_ ^ _08_);
  assign sum[2] = _14_ ^ _05_;
  assign _16_ = ~(_14_ | _05_);
  assign _17_ = _16_ | ~(_02_);
  assign sum[3] = _17_ ^ _01_;
endmodule
```

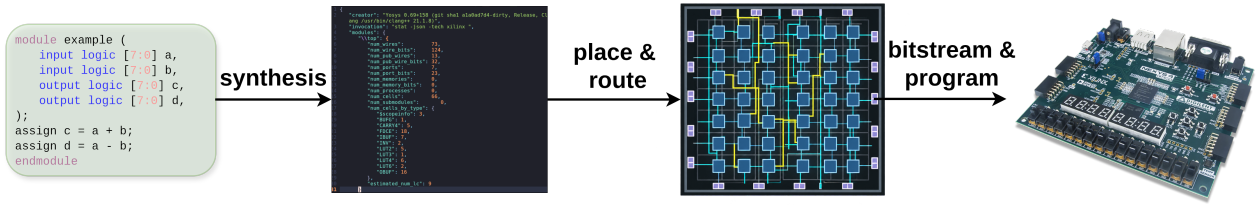
Netlist after initial synthesis

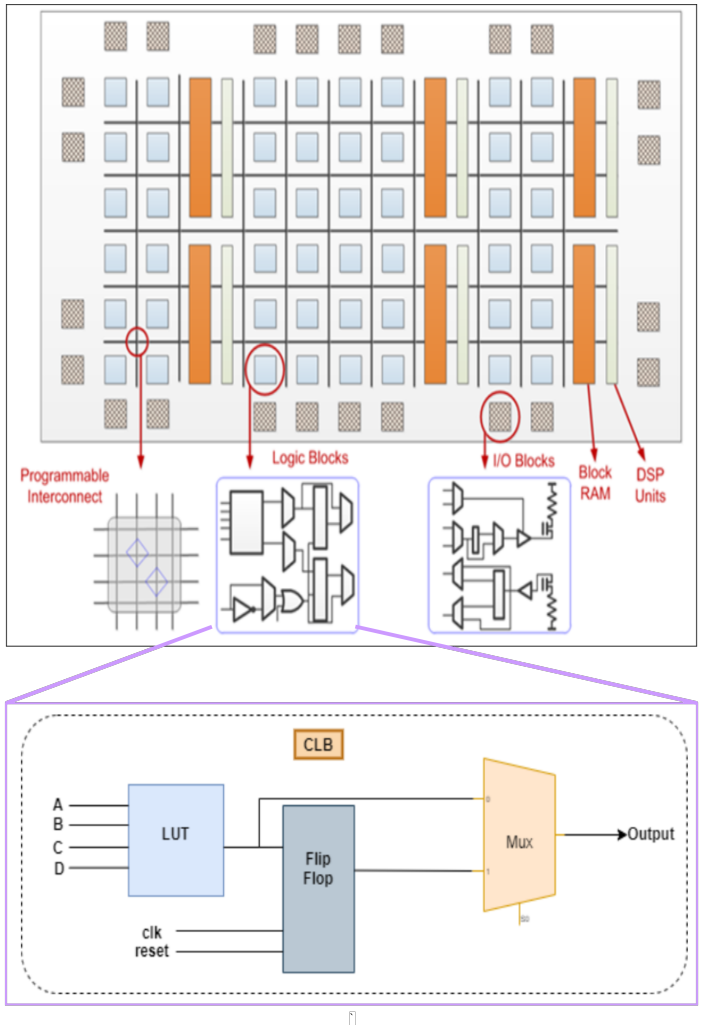
```
module fulladd(a, b, c_in, c_out, sum);
  wire _00_;
  wire _01_;
  wire _02_;
  ...
  output [3:0] sum;
  wire [3:0] sum;
  sg13g2_xor2_1 _61_ (
    .A(_25_),.B(_21_),.X(_42_)
  );
  sg13g2_o21ai_1 _62_ (
    .A1(_40_),.A2(_41_),.B1(_28_),.Y(_27_)
  );
  sg13g2_xnor2_1 _63_ (
    .A(_26_),.B(_35_),.Y(_43_)
  );
  sg13g2_xnor2_1 _64_ (
    .A(_36_),.B(_38_),.Y(_44_)
  );
  sg13g2_xnor2_1 _65_ (
    .A(_31_),.B(_39_),.Y(_45_)
  );
  ...
endmodule
```

Netlist after technology mapping

FPGA Prototyping

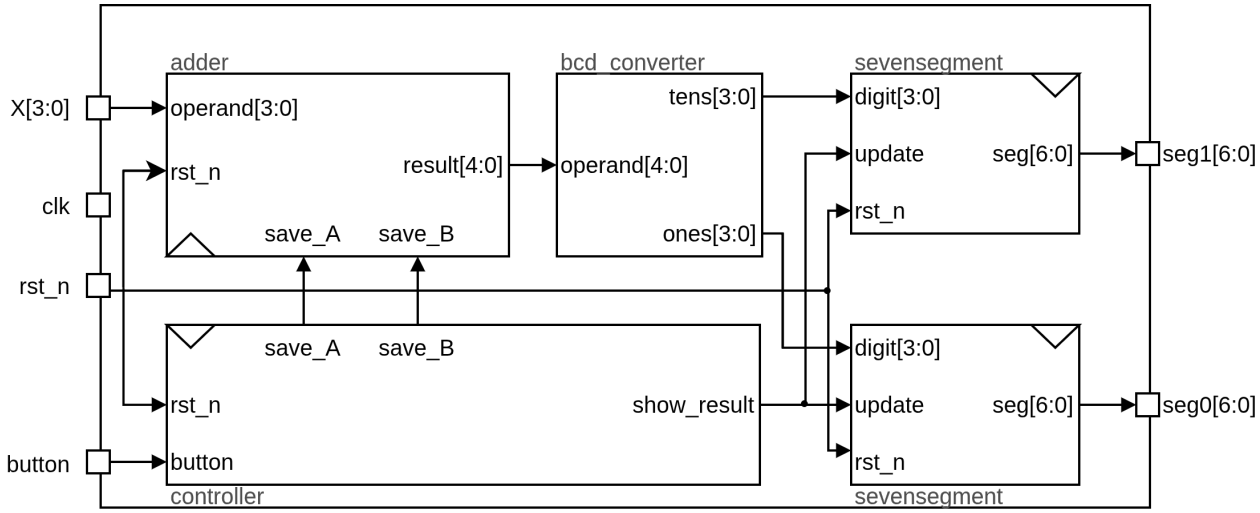
- Integrated circuit with a programmable internal structure
- Contains configurable logic blocks, LUTs, flip-flops, routing resources and dedicated blocks (e.g. memories, DSPs)
- Can be reprogrammed many times





Hands-on Lab: Mini Calculator

Design a small digital system implementing a mini adder controlled by a single button and a finite state machine (FSM).The system sequentially captures two 4-bit operands, computes their sum, and displays the result on two 7-segment displays. The control button is considered ideal, meaning it produces a single-clock-cycle pulse synchronized to clk.



Functional Behavior

The system operates in a sequential manner using a simple FSM and a single control button. The intended usage sequence is:

- The user sets the input bus X[3:0] to operand A
- The user presses the control button $\rightarrow$ operand A is stored
- The user sets the input bus X[3:0] to operand B
- The user presses the control button $\rightarrow$ operand B is stored
- The system computes A + B
- On the next button press, the system display is updated to show the result, and operation returns to step 1

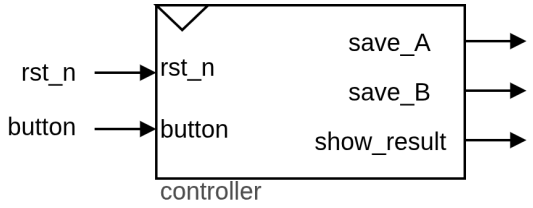
This sequence repeats indefinitely.

Module 1: Controller

FSM sequence

1. GET_A: When button is pressed: capture the current value of X[3:0] into a register, advance to GET_B.
2. GET_B: When button is pressed: capture the current value of X[3:0] into another register, advance to SHOW_RESULT
3. SHOW_RESULT: When button is pressed: change the display value, which is stored in a register, return to GET_A.

The FSM always progresses forward; no backward navigation or editing is required.

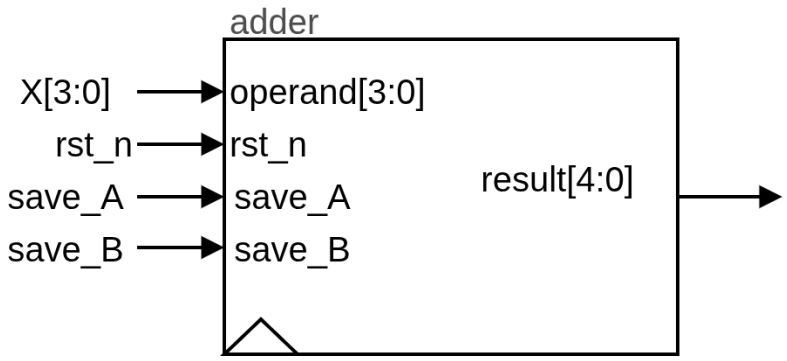


How to: FSM in systemverilog

Module 2: Adder

This Module has to:

1. Set A and B to 0 when ”reset”
2. Save operand to A if signal ”save_A” is active
3. Save operand to B if signal ”save_B” is active
4. Compute the sum A + B into ”result”

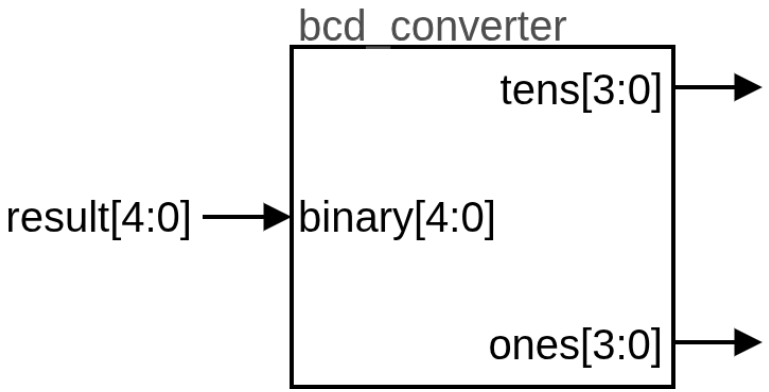


How to: Operators in systemverilog

Module 3: BCD

This Module has to:

1. Separate units and tens of the input number

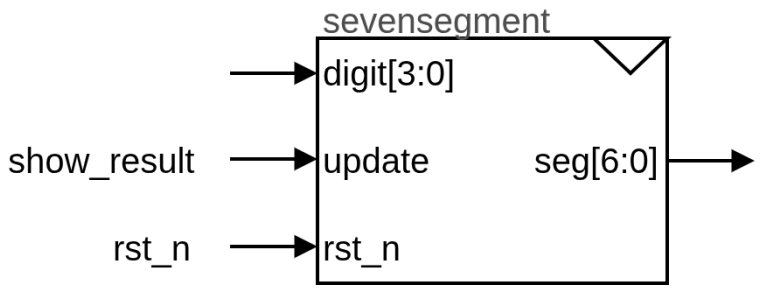


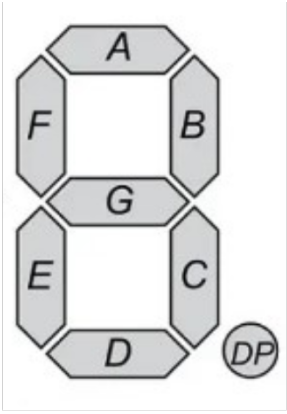
How to: Operators in systemverilog

Module 4: 7segment

This Module has to:

1. Convert binary number into a 7 segment display: a,b,c,d,e,f,g
2. When ”reset” set ”seg” to 0
3. When ”update” save the ”digit”
4. The 7 segment has inverted logic (active low)

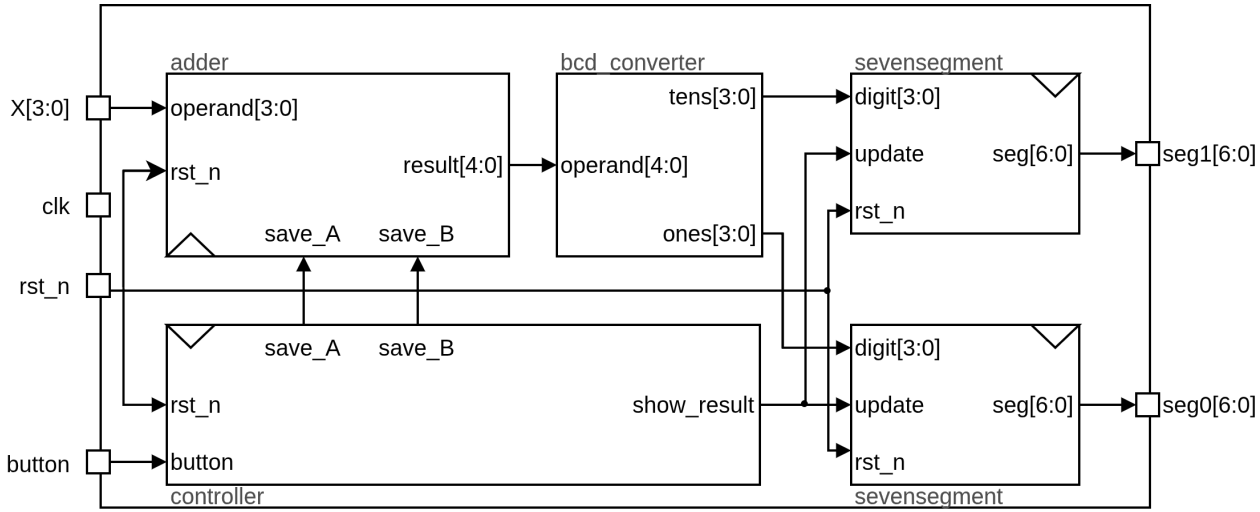




How to: Demultiplexer in verilog

Top Module: mini_calculator

1. Instantiate all the submodule into the top file.

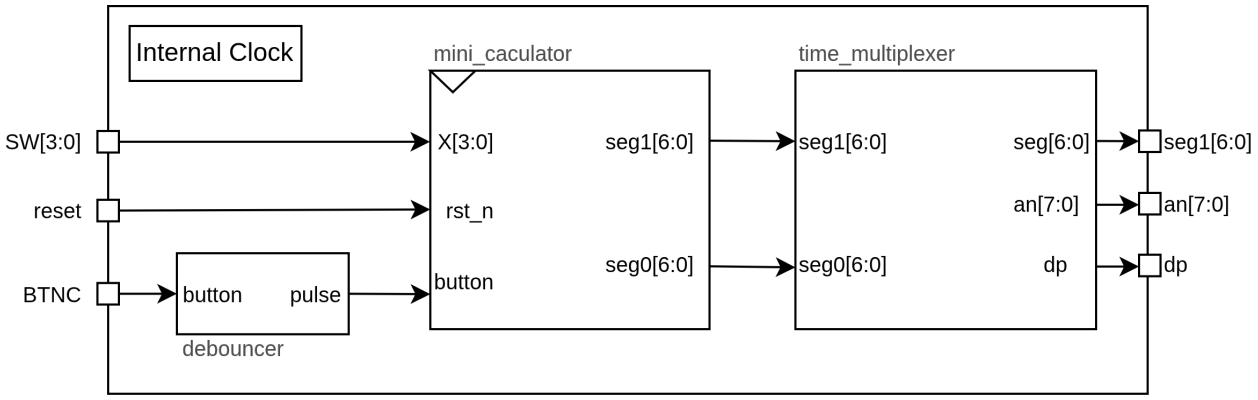


How to: Instantiate modules in verilog

FPGA Implementation

Our mini calculator works in simulation, but real hardware introduces additional challenges:

1. Push Button - Debouncing: How to debounce
2. 7-segment Display - Time Multiplexing: Display time multiplexing in nexys FPGA



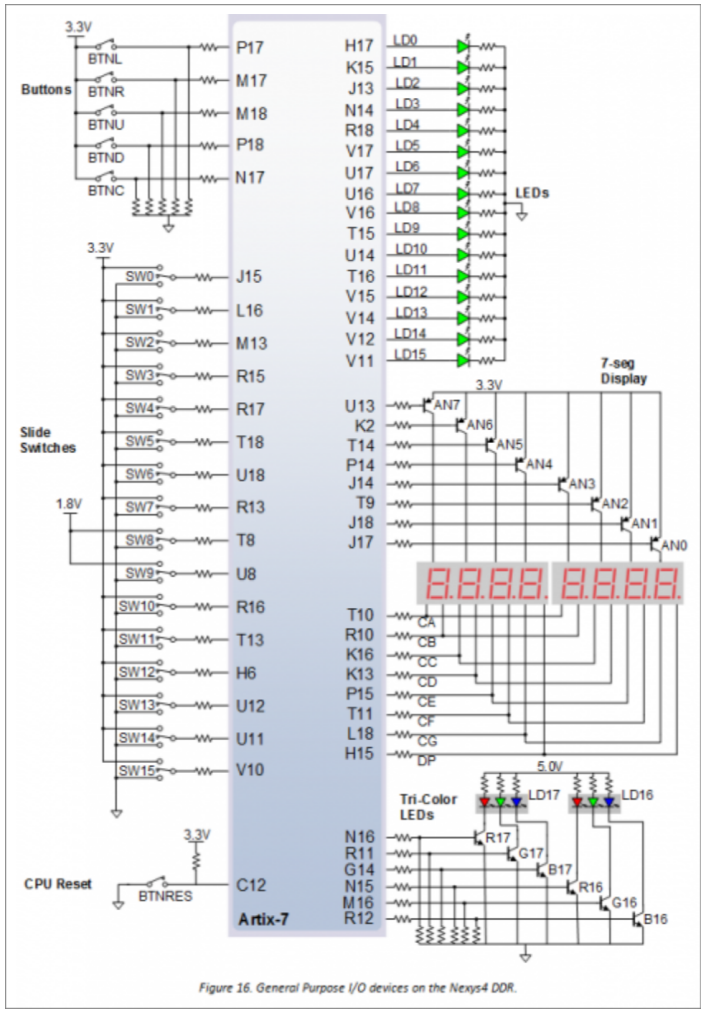


Figure 16. General Purpose I/O devices on the Nexys4 DDR.